Infraestructura: Railway + Netlify + Supabase — Análisis de Performance

Fecha: 2026-04-10
Contexto: Migración del stack actual (Heroku + PHP) al stack propuesto

---

Volumen de datos actual (producción)

Tabla	Registros
`users`	54
`employees`	1,282
`vehicles`	389
`client_branch_offices`	197
`travel_counts`	**1,242,422**
`ttpn_bookings`	**1,535,636**

Las dos tablas críticas superan el millón de registros. El trigger `buscar_booking` escanea `ttpn_bookings` en cada INSERT de travel_count. El performance de la BD es el punto más delicado del sistema.

Costo actual en Heroku (desglose real)

App Heroku	Dyno	Add-ons	Total/mes
`ttpngas` (Rails API principal)	Performance-M $252.88	Rails Autoscale $18 + Redis $3 + Postgres essential $5 + **Postgres standard-0 $50**	**$328.88**
`ttpn` — Ruby web `ttpn.com.mx`	Basic $7	Postgres essential $5	$12.00
`pacific-river-53404` — PHP proxy (Fase 1 móvil → Supabase)	Basic $7	Postgres essential $5	$12.00
`ttpnplaystore` — PHP proxy Android (original)	Basic $7	—	$7.00
`webtulpe` — Ruby web `tulpe.com.mx`	Basic $7	Postgres essential $5	$12.00
Total Heroku			**$371.88/mes**

Observaciones del desglose:

- El dyno `Performance-M` de `ttpngas` cuesta **$250/mes** solo porque Heroku no tiene tier intermedio entre Basic ($7) y Performance ($250). Railway resuelve esto con cómputo gradual.
- `heroku-postgresql:standard-0` ($50/mes) es la BD de producción de Rails. Se migra a Supabase Pro ($25/mes) — misma potencia, mitad de precio.

Estado de cada app legacy:

App	Qué es	Estado
`ttpn`	Ruby web — `ttpn.com.mx`	Mantener mientras el sitio esté activo. Evaluar migrar a Railway si el sitio crece.
`webtulpe`	Ruby web — `tulpe.com.mx`	Mantener mientras el sitio esté activo. Evaluar migrar a Railway.
`pacific-river-53404`	PHP proxy — **Fase 1 de migración móvil**	Activo temporalmente para redirigir app Android a Supabase. **Eliminar al finalizar Fase 2.**
`ttpnplaystore`	PHP proxy original de la app Android	**Eliminar al finalizar Fase 2** — reemplazado por `pacific-river-53404` y luego por Rails API.

Stack propuesto vs Stack actual

Componente	Actual (Heroku)	Propuesto	Ahorro estimado
Rails API (`ttpngas`)	Performance-M $252.88	**Railway Pro $20 + cómputo**	~$220/mes
BD producción	Postgres standard-0 $50	**Supabase Pro $25 + egress**	~$23/mes
Redis	heroku-redis:mini $3	**Railway add-on ~$3/mes**	$0
Rails Autoscale	$18	**No necesario en Railway**	$18/mes
N8N (nuevo)	No existe	**Railway mismo plan**	$0 extra
Sidekiq	Dyno separado	**Railway mismo proyecto**	$0 extra
Frontend Quasar PWA	No existe	**Netlify gratis**	$0
PHP proxy	$7 (temporal)	**Eliminar**	$7/mes
Total	**$371.88/mes**	**~$55–70/mes**	**~$300/mes**

Ahorro anual estimado: ~$3,600 USD.

⚠ Cómo funciona el cómputo en Railway

Railway no cobra tarifa fija pura — cobra **uso real de CPU y RAM**:

```
Plan Pro: $20/mes base (incluye $5 de crédito de cómputo)
```

```
Cómputo adicional:
  CPU:  $0.000463 / vCPU-minuto
  RAM:  $0.0000018 / MB-minuto

Ejemplo Rails API 24/7 con 1 vCPU + 512MB RAM:
  CPU:  0.000463 × 60 × 24 × 30 = ~$20/mes
  RAM:  0.0000018 × 512 × 60 × 24 × 30 = ~$4/mes
  Subtotal cómputo: ~$24/mes + $20 base = ~$44/mes para Rails solo

Con Sidekiq + Redis + N8N (3 servicios adicionales):
  ~$55-70/mes total estimado
```

Esto sigue siendo **5–6 veces más barato** que Heroku Performance-M.

⚠ Cómo funciona el egress en Supabase Pro

```
Plan Pro: $25/mes base
Incluye: 8GB storage, 50GB egress/mes

Egress adicional: $0.09/GB
Storage adicional: $0.125/GB

Con 1.2M travel_counts + 1.5M bookings (~2-5 GB de datos):
  Storage: cubierto en el plan base
  Egress (queries + API): monitorear primer mes — estimado dentro del plan
```

¿El performance sería óptimo, mejor o igual que app nativa?

Respuesta directa: **Igual o mejor** para el 95% de los casos de uso de TTPN

La razón: el cuello de botella en una app como TTPN no es el renderizado (nativo vs PWA), sino la **latencia de red y el tiempo de respuesta del API**. Al eliminar los cold starts de Heroku y mover a Railway siempre activo, el tiempo de respuesta percibido mejora para todos los usuarios, sin importar si usan la app nativa o la PWA.

Análisis de latencia por operación

Heroku Free/Hobby actual — el problema real

```
Primera request después de 30 min de inactividad:
  Cold start: 5-15 segundos de espera

Requests normales:
  API response: 200-800ms (servidor compartido, variable)
```

El chofer que abre la app a las 6am después de que el servidor durmió espera **hasta 15 segundos** para el primer login. Eso es el problema real hoy, no native vs PWA.

Railway $25/mes — siempre activo

```
Primera request (cualquier hora):
  Sin cold start: 0ms de penalización

API response típica con Railway + Supabase Pro:
  Login:                    50–120ms
  GET client_branch_offices: 30–80ms  (197 registros, cached)
  POST travel_count:        100–300ms (trigger buscar_booking sobre 1.5M
rows)
  GET bookings del chofer:  80–200ms  (filtrado por employee_id + fecha)
```

El trigger `buscar_booking` es el más costoso — pero los índices ya están creados en las migraciones (`20260408000001_add_missing_performance_indexes.rb`). Con Supabase Pro (instancia dedicada, más RAM para cache de índices), esa query debería quedar consistentemente bajo 200ms.

PWA vs App nativa — comparación real para TTPN

Operación	App nativa (Android)	PWA en Chrome	Diferencia
Renderizar formulario travel_count	~30ms	~40ms	+10ms — imperceptible
Cálculo Haversine (GPS)	~1ms	~2ms	Imperceptible
POST travel_count al API	depende de red	depende de red	**Idéntico**
Mostrar lista de bookings	~50ms	~60ms	Imperceptible
Abrir cámara para foto	~200ms	~300ms	Leve, no molesta
Animaciones y scroll	Nativo suave	Suave en Android moderno	Equiparable en Android 10+

Conclusión: Para una app orientada a formularios y consultas HTTP, la diferencia nativo vs PWA es de 10–50ms por operación — el usuario no lo percibe. Lo que sí percibe es el tiempo de respuesta del servidor (que mejora con Railway) y los cold starts (que desaparecen).

Impacto por tipo de usuario

Choferes (usuarios móviles — ~30–80 simultáneos en hora pico)

Escenario	Heroku actual	Railway + Supabase Pro
Login a las 6am (cold start)	5–15 seg	< 1 seg

Escenario	Heroku actual	Railway + Supabase Pro
Captura de travel_count	300–800ms	100–250ms
Consulta de booking asignado	200–600ms	80–180ms
Carga de gasolina con GPS	400–900ms	150–300ms

Ganancia real: El chofer percibe una app significativamente más rápida, independientemente de si es nativa o PWA.

Administradores y capturistas (usuarios web — ~5–20 simultáneos)

Escenario	Heroku actual	Netlify + Railway
Cargar panel admin	2–8 seg (cold start)	< 1 seg (Netlify CDN)
Consultas de cuadre (tablas grandes)	variable	consistente
Reportes con filtros	lento sin índices	rápido con índices en Supabase Pro

---

Supabase Pro — qué cambia específicamente

Característica	Free	Pro $25/mes
RAM PostgreSQL	1 GB	8 GB
CPU	Compartido	Dedicado
Conexiones simultáneas	60	200 (+ PgBouncer sin límite)
Storage	500 MB	8 GB
Cache de índices en RAM	Mínimo	Suficiente para 1.5M rows
Backups	Manual	Diario automático
Soporte	Comunidad	Email prioritario

Con 8 GB de RAM, Supabase Pro puede mantener los índices de `travel_counts` y `ttpn_bookings` completamente en memoria — el trigger `buscar_booking` pasa de hacer I/O a disco a operar en RAM. Eso es la diferencia más grande.

Railway — qué cambia específicamente

Característica	Heroku Hobby $7/mes	Railway Pro $25/mes
Cold starts	No (Hobby siempre activo)	No
RAM	512 MB	8 GB
CPU	Compartido limitado	Mejor CPU compartido
Región	US-East	US-West / US-East (elegir más cercano a Chihuahua)

Característica	Heroku Hobby $7/mes	Railway Pro $25/mes
Sidekiq / workers	1 dyno separado	Mismo proyecto, sin costo extra
Deploy	Manual o CI	GitHub auto-deploy
Logs	1,500 líneas	Ilimitado con retención

Región recomendada: `us-west-2` (Oregon) — latencia desde Chihuahua ~40ms vs ~80ms de us-east.

Netlify para el Frontend Quasar

Quasar genera un build estático (`dist/spa/` o `dist/pwa/`). Netlify lo distribuye desde CDN global:

Característica	Detalle
Tiempo de carga inicial (PWA)	< 1 seg (assets en CDN, comprimidos)
Deploy automático	Push a `main` → deploy en ~90 seg
HTTPS	Gratis, automático
Preview URLs por PR	Gratis
Costo	Gratis (Free tier) para este volumen

El Free tier de Netlify es suficiente para TTPN (100 GB de bandwidth/mes, builds ilimitados). El plan $19/mes solo agrega analíticas y más miembros de equipo.

Arquitectura recomendada

```
── FASE 1 (actual/temporal)
──────────────────────────────────────────────────────

App Android
  └─► pacific-river-53404 (PHP · Heroku) ──► PostgreSQL (Supabase Pro)
  └─► ttpnplaystore (PHP · Heroku) ──────────────── (legacy, mismo
destino)

── FASE 2 (objetivo final)
──────────────────────────────────────────────────────

App Móvil (PWA/Capacitor)
  └─► Netlify CDN ──► Rails API (Railway us-west-2) ──► Supabase Pro

Panel Admin / Kumi Admin
  └─► Netlify CDN ──► Rails API (Railway) ──► Supabase Pro

N8N (automatizaciones)
  └─► Railway (mismo proyecto) ──► Rails API via api_keys (M2M)
```

```
  Sidekiq + Redis
    └─► Railway (mismo proyecto) ──► Supabase Pro

── APPS INDEPENDIENTES (siempre)
──────────────────────────────────────────────


  ttpn.com.mx   ──► Heroku Basic (Ruby) ──► Postgres essential
  tulpe.com.mx  ──► Heroku Basic (Ruby) ──► Postgres essential
```

`pacific-river-53404` y `ttpnplaystore` desaparecen al finalizar Fase 2.

Costo mensual total

Servicio	Plan	Costo
Railway (Rails API + Sidekiq)	Pro	$25/mes
Supabase (PostgreSQL)	Pro	$25/mes
Netlify (Frontend)	Free	$0/mes
Redis (Sidekiq)	Railway add-on	~$5/mes
Total		**~$55 USD/mes**

Con el stack actual se paga Heroku Hobby (~$14/mes para Rails + dyno de Sidekiq) + Supabase Free + PHP Heroku. El salto a $55/mes elimina los cold starts, duplica el performance de BD, y da soporte a la migración completa.

Memoria, reinicios y mantenimientos

Problema actual en Heroku — servidor colgado por memoria

En Heroku el dyno no tiene mecanismo automático de reinicio por uso de memoria — cuando Rails/Puma se llena de RAM el proceso se queda colgado y hay que entrar al dashboard a reiniciar manualmente.

¿Puede pasar en Railway? Sí — pero tiene solución automática

Railway permite configurar reinicios automáticos por fallo o por health check fallido, y además se puede agregar un guard dentro de Rails:

Opción 1 — Restart Policy en Railway (configuración del proyecto)

En el dashboard de Railway, cada servicio tiene **Restart Policy**:

```
  Settings → Deploy → Restart Policy
    ├── Always    → reinicia siempre que el proceso termine (crash o OOM)
```

```
    ├── On Failure → reinicia solo si el proceso terminó con error
    └── Never      → no reinicia (no recomendado para producción)
```

Con `Always` o `On Failure`, si Rails/Puma muere por OOM el contenedor se reinicia solo en segundos, sin intervención manual.

Opción 2 — Health Check en Railway

```
Settings → Deploy → Health Check Path: /up
Health Check Timeout: 30s
```

Si el endpoint `/up` no responde en 30 segundos, Railway reinicia el servicio. Rails 7.1 incluye `/up` nativo. Si el servidor se "cuelga" (proceso vivo pero sin responder), el health check lo detecta y fuerza el reinicio.

Opción 3 — `puma_worker_killer` gem (dentro de Rails)

La forma más robusta: Rails se autoreinicia cuando llega a un límite de RAM, sin esperar a que el proceso muera:

```ruby
# Gemfile
gem 'puma_worker_killer'

# config/initializers/puma_worker_killer.rb
PumaWorkerKiller.config do |config|
  config.ram           = 512    # MB máximo por worker
  config.frequency     = 5      # verificar cada 5 segundos
  config.percent_usage = 0.98   # reiniciar worker al 98% de RAM
  config.rolling_restart_frequency = 12 * 3600  # restart total cada 12h
preventivo
end
PumaWorkerKiller.start
```

Cuando un worker Puma supera 512MB, se reinicia solo ese worker — sin downtime, sin intervención.

Comando manual equivalente al reset de Heroku

Si aun así se necesita reiniciar manualmente:

```
# CLI de Railway (equivalente a "restart dyno" en Heroku)
railway service restart

# O desde el dashboard:
# Railway → Proyecto → Servicio → Deployments → ⟳ Restart
```

Comparación

Situación	Heroku	Railway
Servidor colgado por OOM	Reinicio manual desde dashboard	Automático (Health Check o Restart Policy)
Worker lento acumulando RAM	No hay solución nativa	`puma_worker_killer` lo maneja
Reinicio manual urgente	Dashboard → Restart Dyno	`railway service restart` o dashboard
Ver memoria en tiempo real	Métricas básicas	Gráficas de CPU/RAM en tiempo real por servicio

Mantenimientos programados de PostgreSQL

Supabase Pro — ¿cómo maneja los upgrades de Postgres?

Aspecto	Detalle
Versión actual soportada	PostgreSQL 15 y 17
Upgrades de versión mayor	Supabase notifica por email con al menos 7 días de anticipación
Ventana de mantenimiento	Supabase elige horario de bajo tráfico (generalmente madrugada UTC)
Duración del mantenimiento	5–30 minutos típicamente — la BD queda en modo lectura o no disponible
Backup antes del upgrade	Supabase Pro hace snapshot automático antes de cualquier upgrade
Upgrades de parches (15.x → 15.y)	Transparentes, sin downtime perceptible
Auto-pause	**Desactivado en Pro** — la BD nunca se apaga por inactividad
Point-in-time recovery	Incluido en Pro — restaurar a cualquier segundo de los últimos 7 días
Cómo prepararse	El equipo recibe email → planificar mantenimiento de la app en esa ventana

Acción recomendada: Al recibir el email de mantenimiento de Supabase, programar una ventana de mantenimiento en la app (página de "Volvemos pronto") durante esos 30 minutos.

Railway — ¿cómo maneja el mantenimiento de la plataforma?

Railway no gestiona PostgreSQL directamente en este stack (la BD vive en Supabase). Para la plataforma Railway (los contenedores de Rails, Sidekiq, N8N):

Aspecto	Detalle

Aspecto	Detalle
Mantenimiento de plataforma	Railway hace rolling updates — sin downtime para los servicios
Updates del runtime (Ruby, Node)	Responsabilidad del equipo — actualizar `Dockerfile` o `nixpacks.toml`
Updates de la imagen base	Se controlan con el `Dockerfile` — Railway no cambia la imagen sin que tú hagas deploy
Notificaciones	Railway publica en `status.railway.app` y envía email si hay incidentes
SLA Pro	99.9% uptime garantizado

Redis en Railway — mantenimiento

```
Railway gestiona Redis como add-on:
  - Updates de parches: automáticos, transparentes
  - Upgrades de versión mayor: notificación previa, reinicio breve
  - Persistencia: RDB snapshots cada hora (incluido en el plan)
```

Checklist de configuración para evitar colgadas en Railway

```
[ ] Agregar gem 'puma_worker_killer' con límite de RAM configurado
[ ] Configurar Health Check en Railway: /up con timeout 30s
[ ] Configurar Restart Policy: Always (o On Failure)
[ ] Configurar alertas de memoria en Railway dashboard
[ ] Suscribir email del equipo a status.railway.app
[ ] Suscribir email del equipo a notificaciones de Supabase (dashboard →
Settings → Notifications)
[ ] Documentar comando de reinicio manual: `railway service restart`
```

Veredicto final

Para la plantilla de empleados y usuarios de TTPN usando FE web + app móvil (PWA o nativa):

Comparación	Resultado
Railway + Supabase Pro vs Heroku + Supabase Free	**Significativamente mejor**
PWA en Railway vs App nativa en Heroku	**Mejor** (infraestructura domina sobre framework)
PWA en Railway vs App nativa en Railway	**Prácticamente igual** para formularios y consultas

La variable que más importa no es nativa vs PWA, sino Heroku con cold starts vs Railway siempre activo. Un chofer usando una PWA en Railway tendrá mejor experiencia que uno usando la app nativa actual en Heroku.